

ROBOTICS · LESSON 9

Which Way Am I Facing?

Wheels can't tell you which way you point after a slip — a gyro can. Add an IMU, integrate its turn rate into a heading, and fix the turn drift odometry can't.

ROBO-009 · ~45 MIN · INTERMEDIATE

1. The one idea

A gyroscope gives you **heading**. Integrate its turn rate over time into an angle, and the robot finally knows which way it is pointing — correcting the turn drift that wheel ticks alone can never catch.

2. Why it matters

Lesson 7's odometry drifts worst on turns: a wheel slips during a spin and the angle is wrong, and that error compounds along a path. Wheel ticks measure how far the wheels *rolled*, not which way the robot *faced*. A gyro measures rotation directly. Heading is half of knowing where you are — and every drone, phone, and self-driving car fuses an IMU for exactly this.

3. Parts & wiring

- The Lesson 7 rover
- An MPU-6050 (6-axis gyro + accelerometer, I2C) — cheap and everywhere

MPU-6050	ESP32	Why
VCC	3V3	the module's regulator runs from 3.3V
GND	GND	shared reference
SDA	GPIO16	I2C data — remapped clear of the rover's pins
SCL	GPIO17	I2C clock

The default ESP32 I2C pins (21/22) collide with the ultrasonic ECHO on the rover, so we put the IMU on a custom bus with `Wire.begin(16, 17)`. ESP32 I2C can live on almost any two pins — pick ones nothing else is using.

4. Predict (do this before uploading)

- Command a 90° turn using the gyro instead of wheel ticks — more accurate, or less?
- Spin the robot in place ten full turns and stop. Will the heading read exactly 0 again?
- Leave it sitting perfectly still for a minute. Does the heading stay put?

5. Build it — integrate the turn rate

```
#include <Wire.h>

const int MPU = 0x68;
float heading = 0, gyroBiasZ = 0;
unsigned long lastUs = 0;

float readGyroZ() { // turn rate about the vertical axis,
deg/s
    Wire.beginTransaction(MPU);
    Wire.write(0x47); // GYRO_ZOUT_H
    Wire.endTransmission(false); // repeated start
    Wire.requestFrom(MPU, 2);
    uint8_t hi = Wire.read();
    uint8_t lo = Wire.read();
    int16_t raw = (hi << 8) | lo;
    return raw / 131.0; // 131 LSB per deg/s at the +-250 range
}

void setup() {
    Serial.begin(115200);
    Wire.begin(16, 17); // SDA=16, SCL=17
    Wire.beginTransaction(MPU);
    Wire.write(0x6B); Wire.write(0); // PWR_MGMT_1 = 0: wake the MPU-6050
    Wire.endTransmission();

    for (int i = 0; i < 500; i++) { // learn the resting bias
        gyroBiasZ += readGyroZ();
        delay(2);
    }
    gyroBiasZ /= 500;
    lastUs = micros();
}

void loop() {
    unsigned long now = micros();
    float dt = (now - lastUs) / 1e6;
    lastUs = now;
```

```
heading += (readGyroZ() - gyroBiasZ) * dt; // integrate rate → angle
Serial.printf("heading = %.1f deg\n", heading);
delay(20);
}
```

Two ideas, one lesson. First, the gyro reports a *rate* (degrees per second), and you get an *angle* by adding up $\text{rate} \times \text{time}$ — integration. Second, a gyro that reads a tiny non-zero rate while still will slowly creep, so you measure that resting **bias** once and subtract it. Even then, the heading drifts over long runs — that honesty is part of the lesson.

6. Test against your prediction

Open the Serial Monitor at 115200 baud. Rotate the robot 90° by hand — the heading reads about 90. Spin it back — about 0. Now set it down and watch: even with the bias removed, the number creeps slowly. Gyro heading is excellent over seconds to minutes, and untrustworthy over an hour — exactly the trade your §4 predictions should have hit.

7. What goes wrong (pre-loaded debugging)

- **Heading creeps fast while sitting still** → bias not subtracted, or the chip warmed up since you measured it. Re-measure the bias; some creep always remains.
- **No data / garbage** → SDA/SCL swapped, the address is 0x69 not 0x68 (AD0 pulled high), or you forgot to wake the MPU (write 0 to 0x6B).
- **Heading runs the wrong way** → sign or axis. Negate the rate, or confirm you're reading `GYRO_ZOUT` (yaw).
- **Jumpy heading** → integrating too slowly (large dt) or table vibration. Read faster, add light filtering.
- **Drifts badly over a long run** → the honest limit: gyro yaw has no absolute reference. A magnetometer or an external fix gives true north.

8. Checkpoint

The robot knows which way it is pointing — turn rate integrated into a heading, resting bias removed. Turns can now be commanded by *angle reached*, not wheel ticks, which fixes odometry's worst drift. Heading is half of knowing where you are; position is the other half, and Lesson 10 puts them together.

9. Push further

- `turnTo(target)` : spin until the heading hits a target angle — accurate turns at last.
- Fuse gyro heading with wheel odometry (a complementary filter) for the best of both.
- Add a magnetometer (QMC5883) for an absolute heading that doesn't drift.